From prompt to *dependable* workflow.

Use AI for judgment.
Use exact code for everything that must be checked.

Executable Markdown that brings prompts, code, tools and people into one readable file.

Readable by people **Runnable by SPL** **Replayable later**

EXAMPLE / INVOICE TO CHECKED DATA

01 **Read the document**
Exact file I/O

02 **Understand the contents**
Model + declared schema

03 **Check the numbers**
Assertions + integer arithmetic

OUTPUT / Validated JSON, not just a plausible answer

A good prompt is a start. A workflow needs *more*.

SPL closes the gap between a convincing answer and a usable result.

01 **Too much copy, paste, repeat**

Context and steps live across chats and scripts.

> **One Markdown file, shared variables, ordered execution.**

02 **Valid-looking, wrong-shaped output**

An answer is not automatically application-ready data.

> **Typed schemas, strict formats and business-rule assertions.**

03 **Agents with unclear boundaries**

A request to “be careful” is not a permission check.

> **Visible runtime restrictions on files, hosts and tools.**

04 **Expensive, hard-to-reproduce reruns**

Running again can change answers or repeat an action.

> **Append-only traces and pinned, offline replay.**

Useful for: invoice extraction / code review / issue triage / release notes / research reports

Write a document. *Run* the workflow.

1 Explain in Markdown

Headings and ordinary prose give people context. They are commentary, not instructions.

2 Be precise in `spl` fences

Read inputs, branch, calculate and validate with Python-like code.

3 Delegate in blockquotes

Natural-language steps act as agents. `@name` references shared bindings; it is not text substitution.

> Fences and blockquotes run in document order, sharing top-level variables and functions. Blockquotes are **not compiled into SPL**.

summarize.md — One document / one shared scope

````
---
extension: semantic-prompt-language
version: 1
---
# Summarize meeting notes

```spl
let notes = read_file('notes.txt')
```

> Summarize @notes in one sentence
> and save it to @summary.

```spl
assert len(summary) < 300 'too long'
success summary
```
````

Keep the predictable parts *predictable.*

Control flow and exact tools do not call a model. Spend model calls on the ambiguous parts.

Ordinary SPL / no model needed

```
type Money = int

def line_total(price: Money, qty: int) -> Money:
    return price * qty

let total: Money = 0
for qty in [1, 2, 3]:
    total += line_total(1250, int(qty))

let receipt = object:
    'total_cents': total
success receipt
```

Familiar control flow

`if`, loops, functions, lambdas and lexical scopes.

Useful data primitives

Strings, lists, slices, dictionaries and JSON utilities.

Real I/O

`read_file`, `write_file`, `web_fetch`, `web_search`, `shell`.

Reusable modules

Import local `.spl` helpers; build reports with the `markdown` standard library.

Script inputs

Read CLI arguments through `$args` and environment values through `$env`.

Clear outcomes

`success` returns a result; `fail` stops explicitly. Logs stay on stderr.

SPL is Python-like, not Python. Blocks use a colon and exactly four spaces per level. Dictionaries use `object:`, or `object()` when empty.

Three ways to use AI.
Choose the right capability.

01 / GENERATE

ask

RETURNS A VALUE / NO TOOLS

Extract, classify, translate or draft when the inputs are already available.

```
let title = ask(len='sentence') \
    'Title for: {draft}'
```

02 / REASON

think

RETURNS A VALUE / NO TOOLS

Plan or analyze with medium reasoning by default; raise or lower the effort.

```
let plan = think(reasoning='high') \
    'Plan migration of {service}'
```

03 / ACT

agent

USES RUNTIME + MCP TOOLS

Inspect files, use external tools and take actions within active restrictions.

```
let report = agent \
    'Inspect tests and report'
```

Choose your backend. OpenAI, Anthropic or Google through official clients. Select a model per document or call; CLI `--model` overrides both.

Control the request. Length hints, reasoning, token limits; `ask` also supports `seed`, `temperature` and `top_p` where supported.

> Sampling is best-effort, not reproducibility. Every call starts a fresh conversation: pass the context it needs.

Make “looks right” *checkable.*

Describe the data you expect

Primitives, enums, exact structs, `list[T]`, unions, transparent aliases and callable signatures.

Keep the contract after creation

Known typed errors stop execution before model calls. Runtime boundaries and mutations remain checked.

Separate data from instances

Structs describe records. Classes add identity and methods; direct schema hydration requires `hydratable class`.

Use exact arithmetic for money

Signed 64-bit `int` is distinct from finite `float`. Use cents and explicit numeric conversions.

Shape + meaning / two different checks

```
type Money = int

struct Invoice:
    vendor: str
    subtotal: Money
    tax: Money
    total: Money

let invoice: Invoice = ask(
    schema=Invoice, repairs=2
) 'Extract invoice amounts in cents: {document}'

assert invoice.subtotal + invoice.tax == invoice.total \
    'totals do not add up'
```

`schema=Invoice` selects typed decoding; an annotation alone does not. Struct fields are required and exact, including nullable fields.

Field metadata is model advice, not a business rule. Use assertions for meaning; valid structure does not prove factual correctness. `object(instance)` creates a deep data snapshot.

Reject bad output. Don't repeat *bad actions.*

01 / Parse

`fmt` checks text, numeric, Boolean, array, object or JSON output.

02 / Validate

`schema` accepts SPL type descriptors or raw JSON Schema.

03 / Repair

`repairs=0..5` permits extra attempts on format or schema errors.

04 / Assert

Exact code checks business rules after generation completes.

A strict format is not a range check

```
let score = ask(
    fmt='integer', repairs=2, tokens=2048
) 'Rate this issue from 1 to 5: {issue}'
assert 1 <= score and score <= 5 \
    'score out of range'
success score
```

Failures are explicit

`try / except` can handle runtime failures with a reason, source line and status code.

No hidden cleanup magic

Assertions are not automatic repair prompts. Tool failures and transport errors are not repaired; agents do not auto-repeat failed actions.

> Failures do not roll back earlier writes. Design side effects deliberately.

Let agents act. Draw the *boundary*.

Built-in tools handle files, web access and shell commands. MCP connects agents to your tool ecosystem.

Runtime permissions / narrower scopes intersect

```
let report = ''
restrict './src', exec 'git', \
    net 'docs.example.com', mcp 'docs/search':
    report = agent \
        'Review source and git diff using docs'
success report
```

Load MCP with `--mcp mcp.json`: VS Code-style configuration for stdio, HTTP or SSE servers, with tool allowlists.

fs

Limit files

Resolved paths, including traversal and symlinks, must stay inside permitted roots.

net

Limit hosts

Network calls and redirect hops are checked against allowed hosts.

exec

Limit commands

Allow executable names; check chains, path-like arguments and redirections.

mcp

Limit integrations

Expose only permitted servers or individual tools to agents.

Know the boundary: tools are allowed by default outside restrictions. Restrictions are not an OS sandbox; allowed programs and MCP servers remain trusted. `enforce` / `forbid` are prompt guidance, not permissions.

Keep people in the loop. Reuse what *works.*

HUMAN INPUT

Ask before moving forward

Use `human` in code, or ask a blockquote agent to consult a person through `ask_human`.

```
let approval = human 'Publish? yes/no'
assert approval == 'yes' 'not approved'
success approval
```

Answer in the terminal, or provide repeated `--human` flags in question order. Input is a string; validate it explicitly.

BLOCKQUOTE CONTROLS

`@CTL(...)` sets scoped restrictions, `enforce`, `forbid`, model and reasoning before the agent starts.

EXPLICIT CODE GENERATION

Generate a function, not an action

`@CODEGEN` produces reusable SPL function definitions for later code and agent steps.

````
> @CODEGEN 'Define double(value), returning value multiplied by two'

```spl
success double(21)
```
````

Generated definitions must parse and pass static contract checks. They are not invoked during generation.

> Traced runs cache matching definitions in sibling `.spl` files. Review generated code before trusting it; a description hash is not an integrity guarantee.

A run you can inspect. A result you can *replay*.

Capture model answers, decisions, tool outcomes and human input instead of asking the world to behave identically twice.

APPEND-ONLY HISTORY

Resume with `--trace`

```
spl flow.md --trace run.jsonl
spl flow.md --trace run.jsonl
```

The same path resumes recorded work or replays a completed result. Use a new trace file for a fresh run.

SELF-CONTAINED RECORDED RUN

Freeze with `--pin`

```
spl flow.md --pin
spl flow.locked.md
```

A successful run creates locked Markdown containing its execution journal. Replay needs no model login or MCP calls.

No duplicate external effects

Recorded calls, writes, commands and prompts are not repeated. Replay does not restore changed or missing external files.

Mismatch is a failure, not a live fallback

Source and execution settings must match. Interrupted effects with unknown outcomes require manual recovery.

> Keep journals and locked files private: they can contain sensitive inputs and environment values. `--pin` and `--trace` cannot be combined.

A small language. A *real* editing experience.

SPL NOTEBOOK / typed-data.md / illustrative cell

```
type Money = int
let unit: Money = 1250
let quantity: int = 2
let total: Money = unit * quantity

assert total == 2500
total
```

```
CELL RESULT
2500
```

Notebook sessions retain variables across explicit cell runs. Editing alone never executes a workflow.

```
edit > --check > run > inspect > refine
```

Catch problems as you type

Parser and type diagnostics, completions, hover and signature help.

Navigate the workflow

Definitions, references and safe rename across Markdown fences and `@name` references; symbols, folding and semantic highlighting.

Run in the format that fits

CLI for workflows; VS Code notebooks for cells, streamed responses and visible errors. `.spl` files support modules and editor scratch work.

Integrate and observe

Embed through `parseMarkdown`, `run` and `AiBackend`; use `ExecutionSession` for retained state. CLI logs expose model/tool events and debug variable changes.

Editor analysis is offline. Notebook execution requires a trusted workspace. The CLI runs Markdown workflows, not standalone `.spl` scratch files.

Start small. Make one useful thing *repeatable*.

INSTALL / NODE.JS 24.15.0 OR NEWER

```
npm install -g spl-lang
spl login openai
# Or: spl login anthropic / spl login google
```

FROM THE REPOSITORY ROOT / NO MODEL OR LOGIN

```
spl lang/examples/typed-data.md
spl lang/examples/basics.md --check
```

THEN RUN AN AI WORKFLOW

```
spl lang/examples/basics.md --trace first-run.jsonl
```

1. **Pick a repetitive task**

   Extract an invoice, triage issues, draft release notes or review a diff.

2. **Separate judgment from certainty**

   Let the model interpret. Let code validate. Restrict actions before using tools.

3. **Keep a trustworthy record**

   Add human review where it matters; trace a run or pin a successful result.

Install the **SPL** VS Code extension and type `splworkflow` to create a new workflow. Login configures a provider API key, not subscription OAuth.

Documentation · Runnable examples · VS Code extension · Language reference · Safety reference · CLI & replay reference